

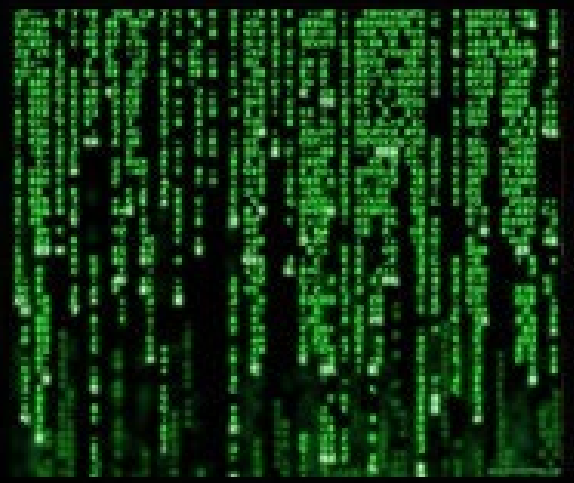
# Introduktion till Programmering

# Dagens föreläsning

1. **Programmering** – Vad är det egentligen?
2. **Programmering**sspråk – Python?
3. **Programmering**smiljö?
    1. VS Code
    2. Terminalen/Konsolen
4. Att börja **programmera**
    1. Hur skriver man?
    2. Var skriver man?
    3. Hur kör man sin kod?
5. Introduktion till datatyper & variabler i Python

# Frågor innan vi börjar?

# PROGRAMMING



WHAT PEOPLE THINK I DO



WHAT MY MOTHER THINKS



WHAT MY FRIENDS THINK I DO



WHAT MY TEACHER THINKS



WHAT I THINK I DO

```
1   Program
2
3   Begin
4
5
6
7   End.
```

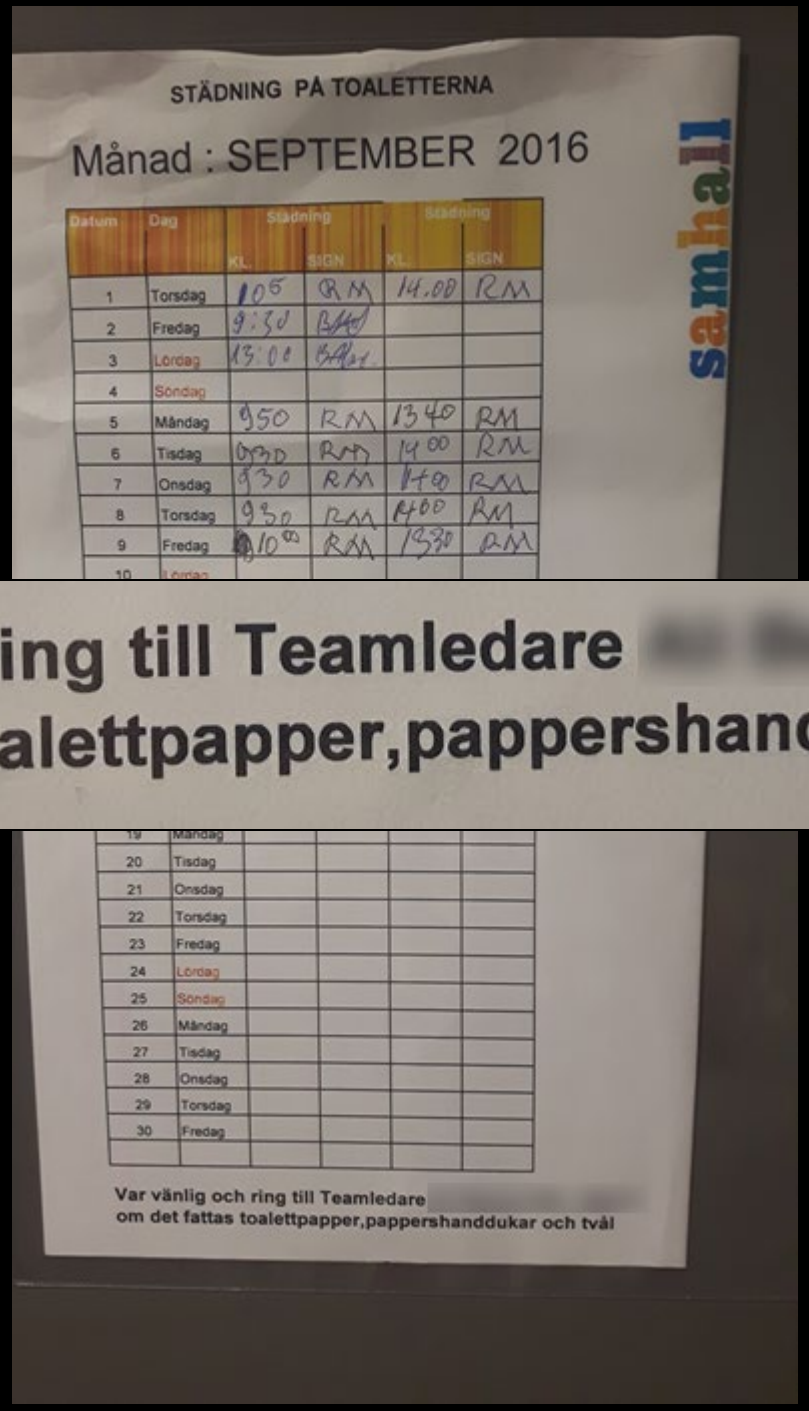
WHAT I ACTUALLY DO

quickmeme.com

# Programmering

- Programmering går ut på att ge instruktioner till dator
- Datorn gör det du säger till den, och inget annat
- Datorn behöver specifika instruktioner





STÄDNING PÅ TOALETTERNA

Månad : SEPTEMBER 2016

samhall

| Datum | Dag | Städning KL. | Städning SIGN | Städning KL. | Städning SIGN |
|---|---|---|---|---|---|
| 1 | Torsdag | 10[illegible] | RM | 14.00 | RM |
| 2 | Fredag | 9:30 | [illegible] | | |
| 3 | Lördag | 13:00 | [illegible] | | |
| 4 | Söndag | | | | |
| 5 | Måndag | 950 | RM | 1340 | RM |
| 6 | Tisdag | [illegible] | RM | 1400 | RM |
| 7 | Onsdag | 930 | RM | 1400 | RM |
| 8 | Torsdag | 930 | RM | 1400 | RM |
| 9 | Fredag | [illegible]10[illegible] | RM | 1330 | RM |
| 10 | Lördag | | | | |
| 19 | Måndag | | | | |
| 20 | Tisdag | | | | |
| 21 | Onsdag | | | | |
| 22 | Torsdag | | | | |
| 23 | Fredag | | | | |
| 24 | Lördag | | | | |
| 25 | Söndag | | | | |
| 26 | Måndag | | | | |
| 27 | Tisdag | | | | |
| 28 | Onsdag | | | | |
| 29 | Torsdag | | | | |
| 30 | Fredag | | | | |
| | | | | | |

Var vänlig och ring till Teamledare [illegible]
om det fattas toalettpapper,pappershanddukar och tvål

Var vänlig och ring till Teamledare [illegible]
om det fattas toalettpapper,pappershanddukar och tvål

# Being a Programmer



**Mom said**: "Please go to the shop and buy 1 bottle of milk. If they have eggs, bring 6

**I came back with 6 bottles of milk.**

**She said**: "Why the hell did you buy 6 bottles of milk?"

“BECAUSE THEY HAD EGGS"

# Exempel på instruktioner

Matematiska instruktioner (beräkningar)

```
>>> 5 + 5
10
>>> 10 * 2
20
>>> 20 / 4
5
```

# Exempel på instruktioner

Utskrift till användaren

```
>>> print("Hello World")
Hello World
>>> print("Tjena kexet, sitter du här och smular?")
Tjena kexet, sitter du här och smular?
```

# Exempel på instruktioner

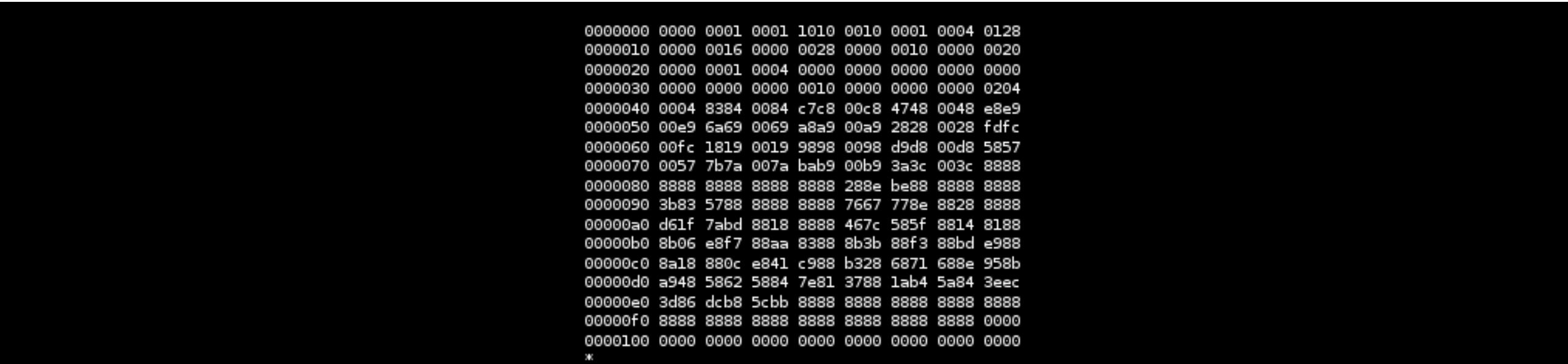
Instruktion att hämta data från användare (input)

```
>>> input("Vad heter du?")
Vad heter du?Anton
'Anton'
>>> input("Vilket är Sveriges bästa fotbollslag?")
Vilket är Sveriges bästa fotbollslag?Elfsborg
'Elfsborg'
```

# Hur fungerar ett program?

- Program är en (ofta väldigt stor) samling av maskininkod – som innehåller instruktioner till datorn

- Det är väldigt ovanligt att en person skriver maskinkod direkt, då detta inte är så enkelt…



# Lågnivå- och högnivåspråk

- Maskinkod var ju inte så enkelt att förstå - men det gör inget!
- Maskinkod är ett **lågnivåspråk**
  - Binärt och hexadecimalt

- Vi kommer att jobbat med **högnivåspråk**
  - T.ex. Python, JavaScript, C#,, C++, Java, PHP, Ruby, etc.

```
# Asks user for their name
name = input("Please enter your name :")
if len(name) < 4:
    # If the users name has less than 4 characters
    print(name + " is a short name")
elif len(name) < 7:
    # If the users name has less than 7 characters (and more than 3)
    print(name + " is a medium long name")
else:
    # If the users name has mote than 6 characters
    print(name + " is a long name")
```

**Resultat (3 körningar)**

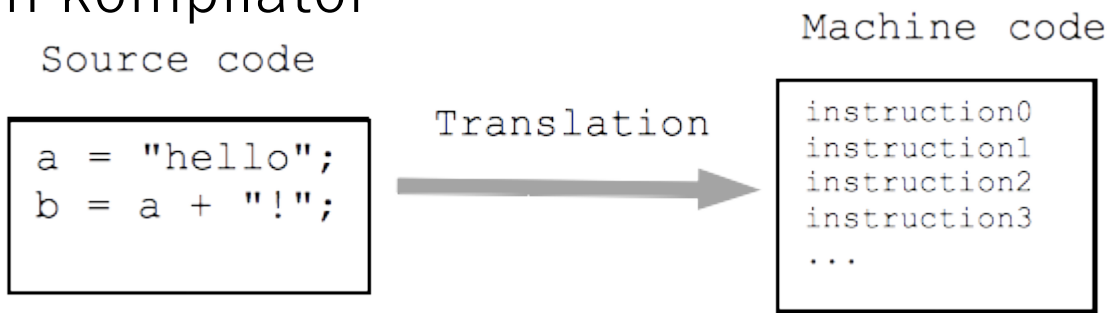
```
>>> ================================ RESTART
>>>
Please enter your name: Anton
Anton is a medium long name!
>>> ================================ RESTART
>>>
Please enter your name: Eva
Eva is a short name!
>>> ================================ RESTART
>>>
Please enter your name: Josefin
Josefin is a long name!
```

```
0000000 0000 0001 0001 1010 0010 0001 0004 0128
0000010 0000 0016 0000 0028 0000 0010 0000 0020
0000020 0000 0001 0004 0000 0000 0000 0000 0000
0000030 0000 0000 0000 0010 0000 0000 0000 0204
0000040 0004 8384 0084 c7c8 00c8 4748 0048 e8e9
0000050 00e9 6a69 0069 a8a9 00a9 2828 0028 fdfc
0000060 00fc 1819 0019 9898 0098 d9d8 00d8 5857
0000070 0057 7b7a 007a bab9 00b9 3a3c 003c 8888
0000080 8888 8888 8888 8888 288e be88 8888 8888
0000090 3b83 5788 8888 8888 7667 778e 8828 8888
00000a0 d61f 7abd 8818 8888 467c 585f 8814 8188
00000b0 8b06 e8f7 88aa 8388 8b3b 88f3 88bd e988
00000c0 8a18 880c e841 c988 b328 6871 688e 958b
00000d0 a948 5862 5884 7e81 3788 1ab4 5a84 3eec
00000e0 3d86 dcb8 5cbb 8888 8888 8888 8888 8888
00000f0 8888 8888 8888 8888 8888 8888 8888 0000
0000100 0000 0000 0000 0000 0000 0000 0000 0000
*
0000130 0000 0000 0000 0000 0000 0000 0000
000013e
```

```python
# Asks user for their name
name = input("Please enter your name :")
if len(name) < 4:
    # If the users name has less than 4 characters
    print(name + " is a short name")
elif len(name) < 7:
    # If the users name has less than 7 characters (and more than 3)
    print(name + " is a medium long name")
else:
    # If the users name has mote than 6 characters
    print(name + " is a long name")
```

# Hur fungerar högnivåspråk?

- Hur kan datorn förstå vår högnivåprogrammering, när den bara förstår binära tal?
  - Vi översätter vår högnivåkod till lågnivå
- Detta kan man göra på två olika sätt:
  - Genom en tolk
  - Genom en kompilator




```
# Asks user for their name
name = input("Please enter your name :")
if len(name) < 4:
    # If the users name has less than 4 characters
    print(name + " is a short name")
elif len(name) < 7:
    # If the users name has less than 7 characters (and more than 3)
    print(name + " is a medium long name")
else:
    # If the users name has mote than 6 characters
    print(name + " is a long name")
```

```
0000000 0000 0001 0001 1010 0010 0001 0004 0128
0000010 0000 0016 0000 0028 0000 0010 0000 0020
0000020 0000 0001 0004 0000 0000 0000 0000 0000
0000030 0000 0000 0000 0010 0000 0000 0000 0204
0000040 0004 8384 0084 c7c8 00c8 4748 0048 e8e9
0000050 00e9 6a69 0069 a8a9 00a9 2828 0028 fdfc
0000060 00fc 1819 0019 9898 0098 d9d8 00d8 5857
```

# Tolkning och kompilering av kod

- Tolkning av kod sker ”on the fly”

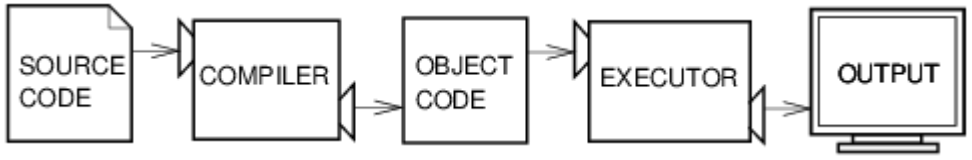




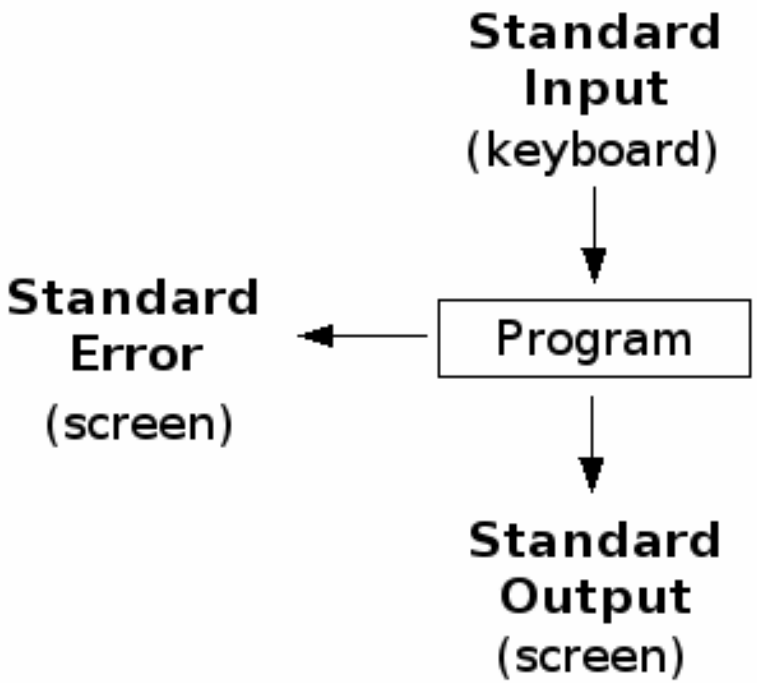


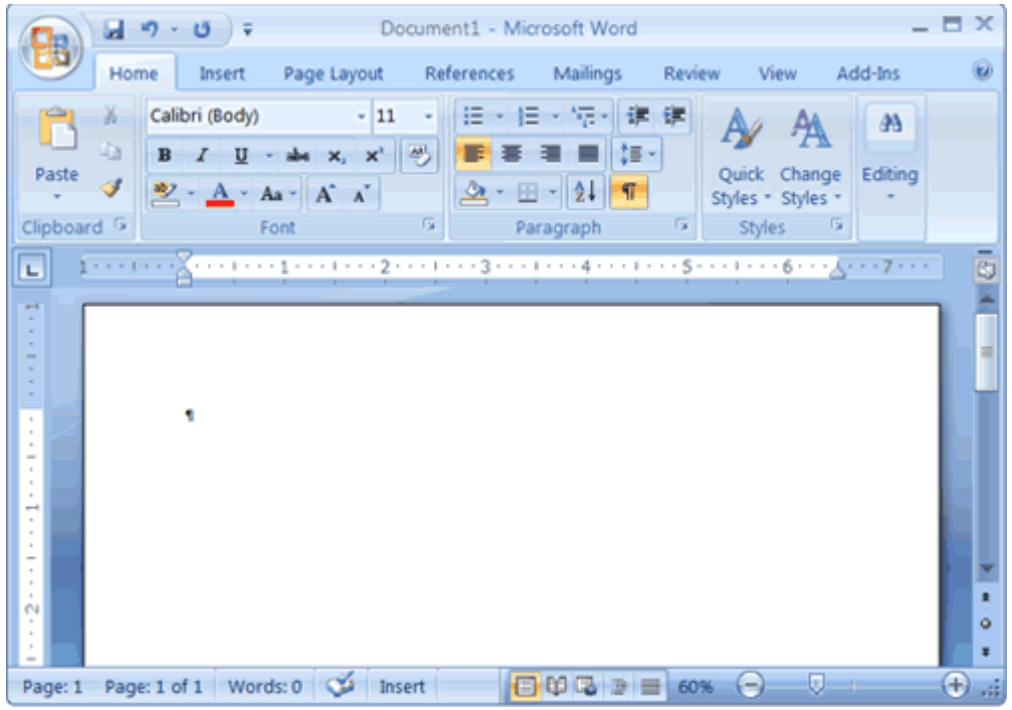


- Kompilering av kod sker ”i förväg”



# Program?


Standard
Input
(keyboard)
Standard
Error
(screen)
Program
Standard
Output
(screen)

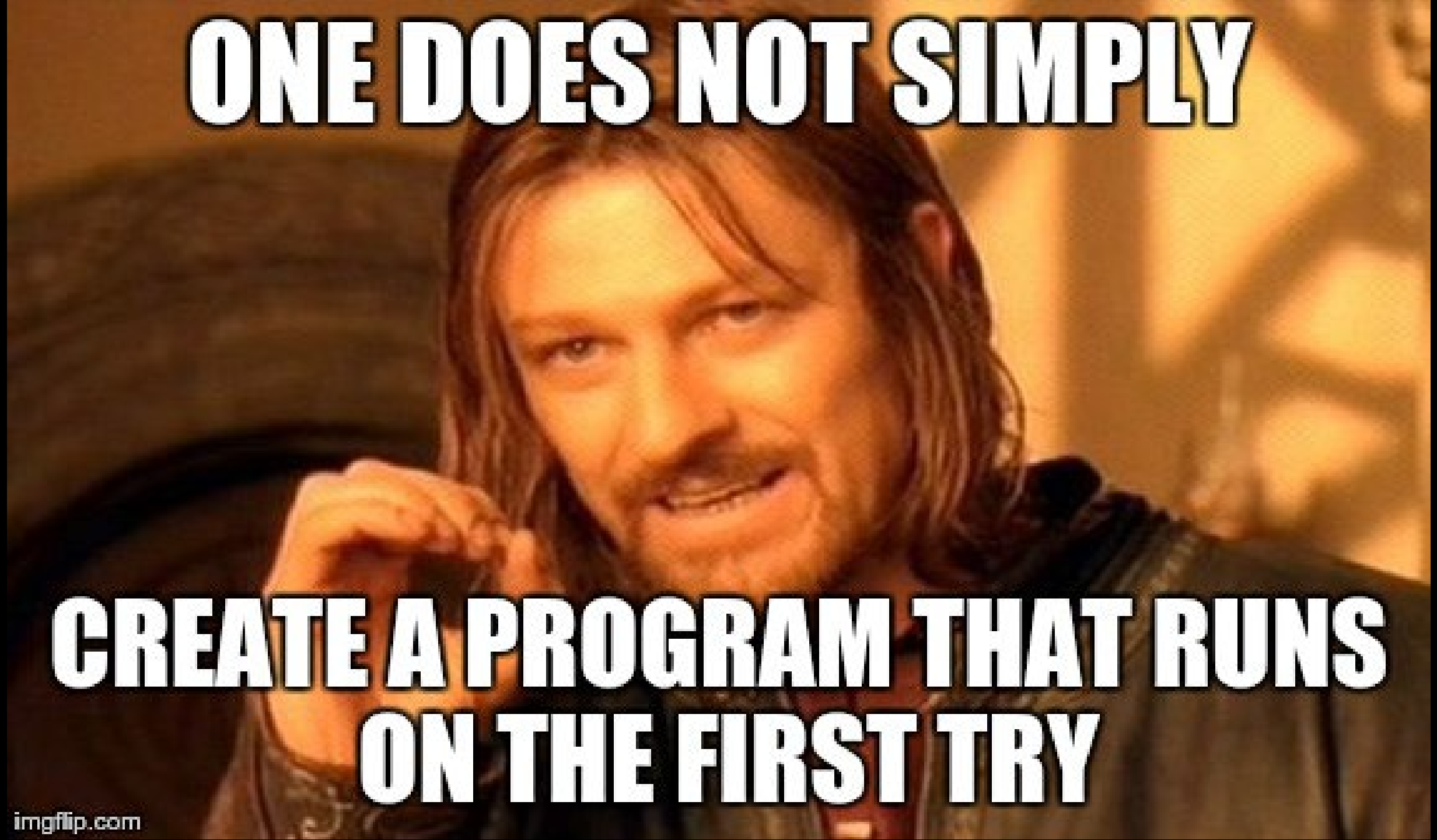

Document1 - Microsoft Word
Home
Insert
Page Layout
References
Mailings
Review
View
Add-Ins
Calibri (Body)
11
Paste
Clipboard
Font
Paragraph
Quick Styles
Change Styles
Styles
Editing
Page: 1
Page: 1 of 1
Words: 0
Insert
60%

# Vad gör ett program egentligen?

| | | |
|---|---|---|
| Input | ⟷ | Från användare |
| Output | ⟷ | Till användare |
| Beräkningar | ⟷ | Beräkningar |
| Konditional exekvering | ⟷ | If-satser |
| Repetition | ⟷ | Iterationer |

# Error?

Men nä, va, hur?


ONE DOES NOT SIMPLY
CREATE A PROGRAM THAT RUNS ON THE FIRST TRY
imgflip.com

# Olika typer av fel

- Syntax – Följer inte programmeringsspråket regler (grammatik)
  - `2 + ”hej”`
  - `print(hej)`
- Runtime error – Fel under programmets körning

```
Traceback (most recent call last):
  File "C:/Users/TSANTI/Desktop/name.py", line 1, in <module>
    print name
NameError: name 'name' is not defined
```

- Semantiska fel

```
Welcome to the addition calculator!
-----------------------------------
Nr 1: 4
Nr 2: 5
Result: 20
```

# Kärlek till programmering



# Debugging

- Att lära sig hantera dessa felmeddelande är en programmerares vardag
  - Ju mer ni programmerar, ju bättre kommer ni att bli på detta
- **Kom ihåg – alla gör fel, mest hela tiden!**

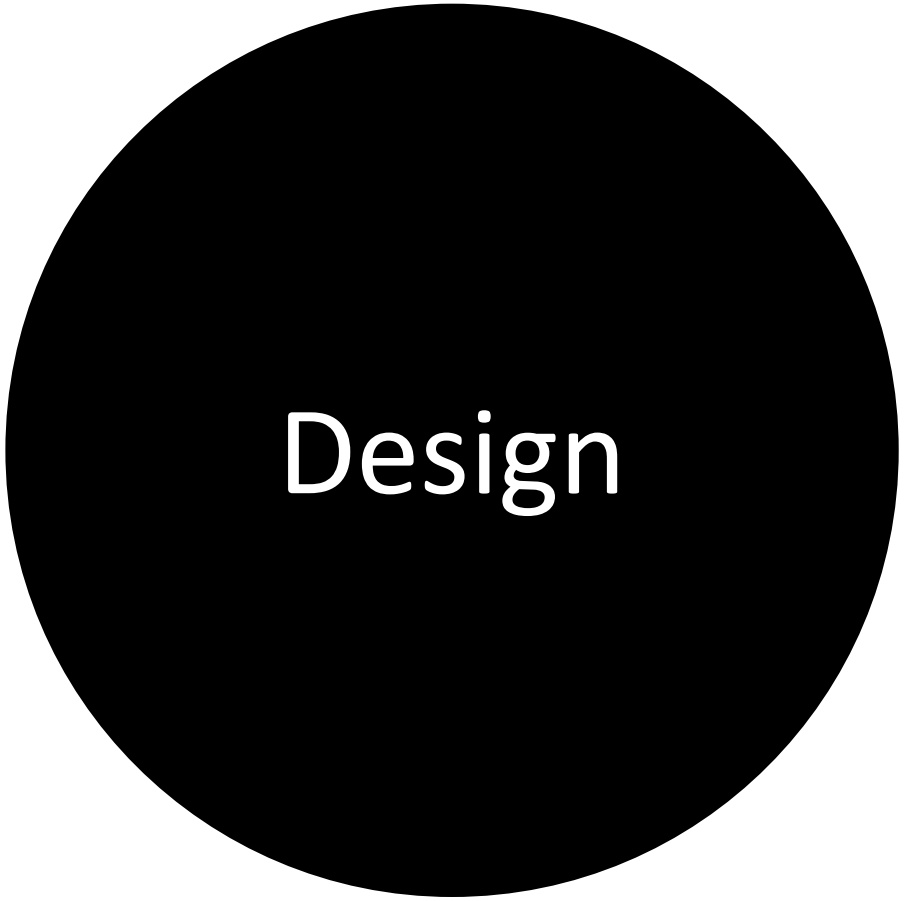
- Tänk på att programmeringsspråk är formella språk – och måste vara exakta
  - Till skillnad från naturliga språk som t.ex. svenska och engelska där vi inte behöver vara exakta för att förstå varandra – vi kan till och med använda slang – och förstå varandra! =)


It doesn't work..... why?
It works..... why?
MemeBlender.com

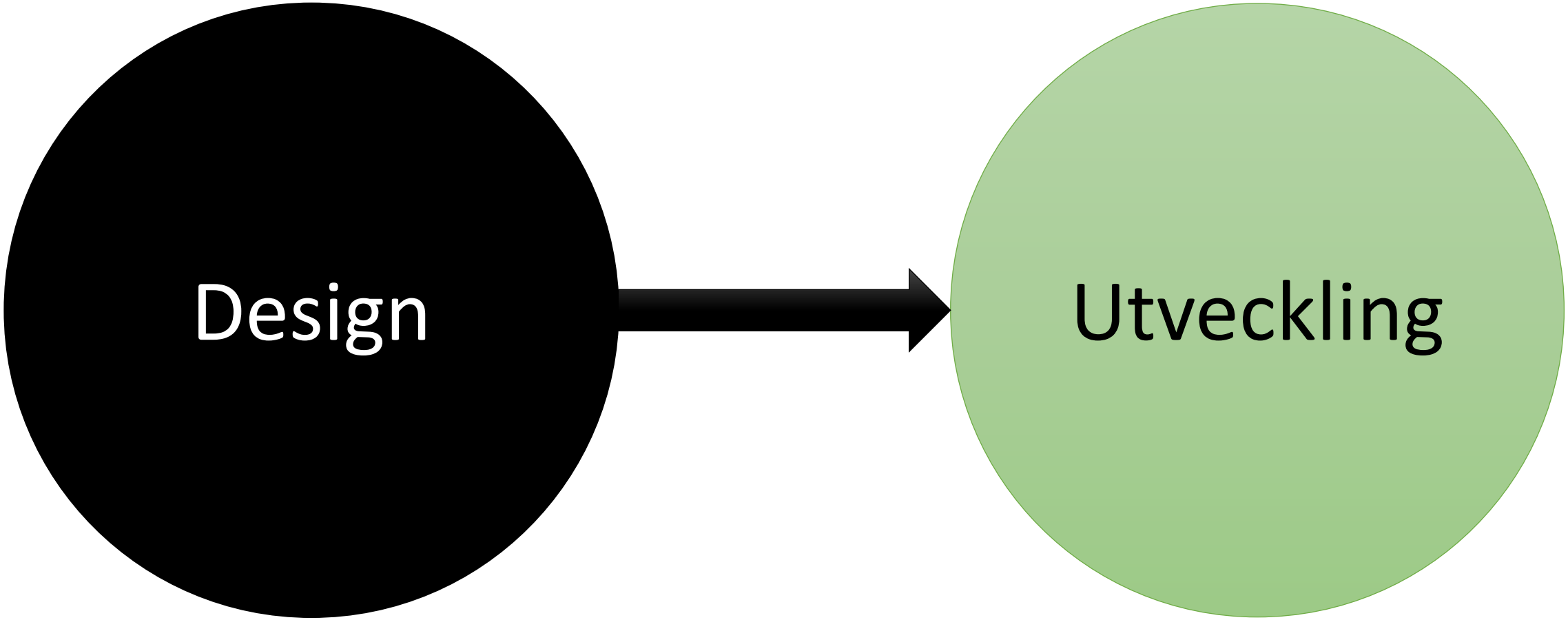
# Varför läser vi programmering?

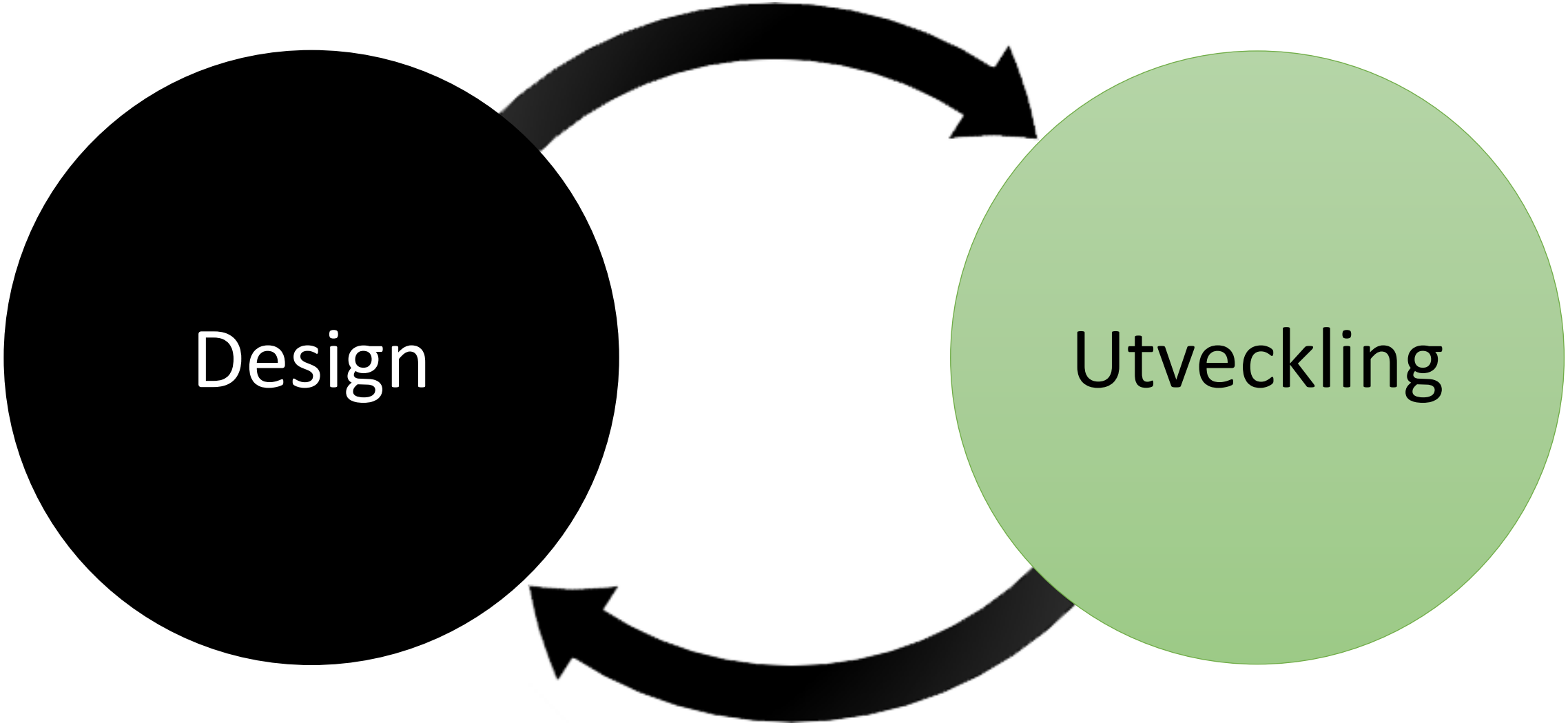
Kan man utforma bra **webbplatser** utan att kunna **webbtekniker**?

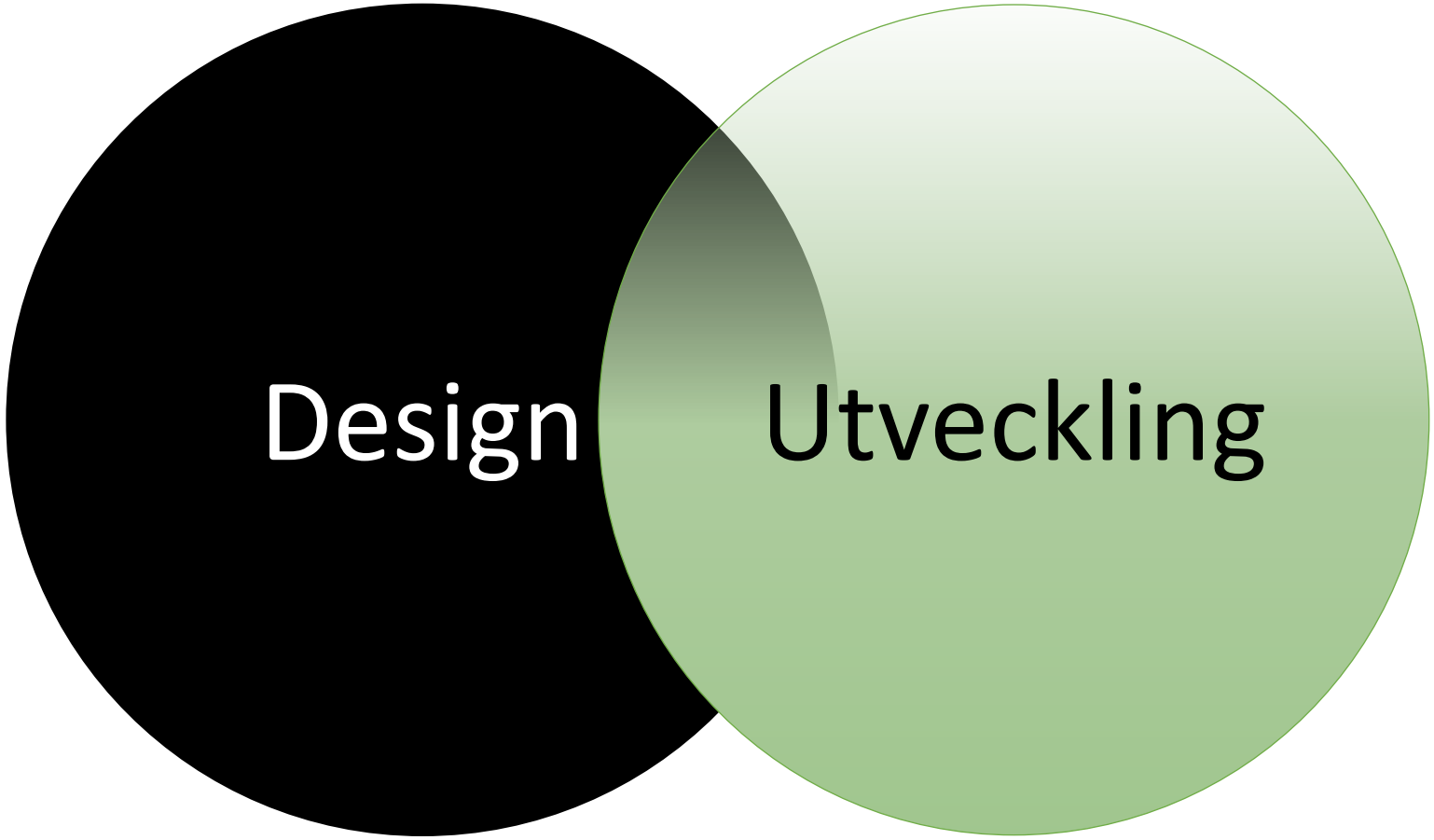
Kan man utforma bra **informationssystem** utan att kunna **programmera**?


Design


Utveckling


Design
Utveckling

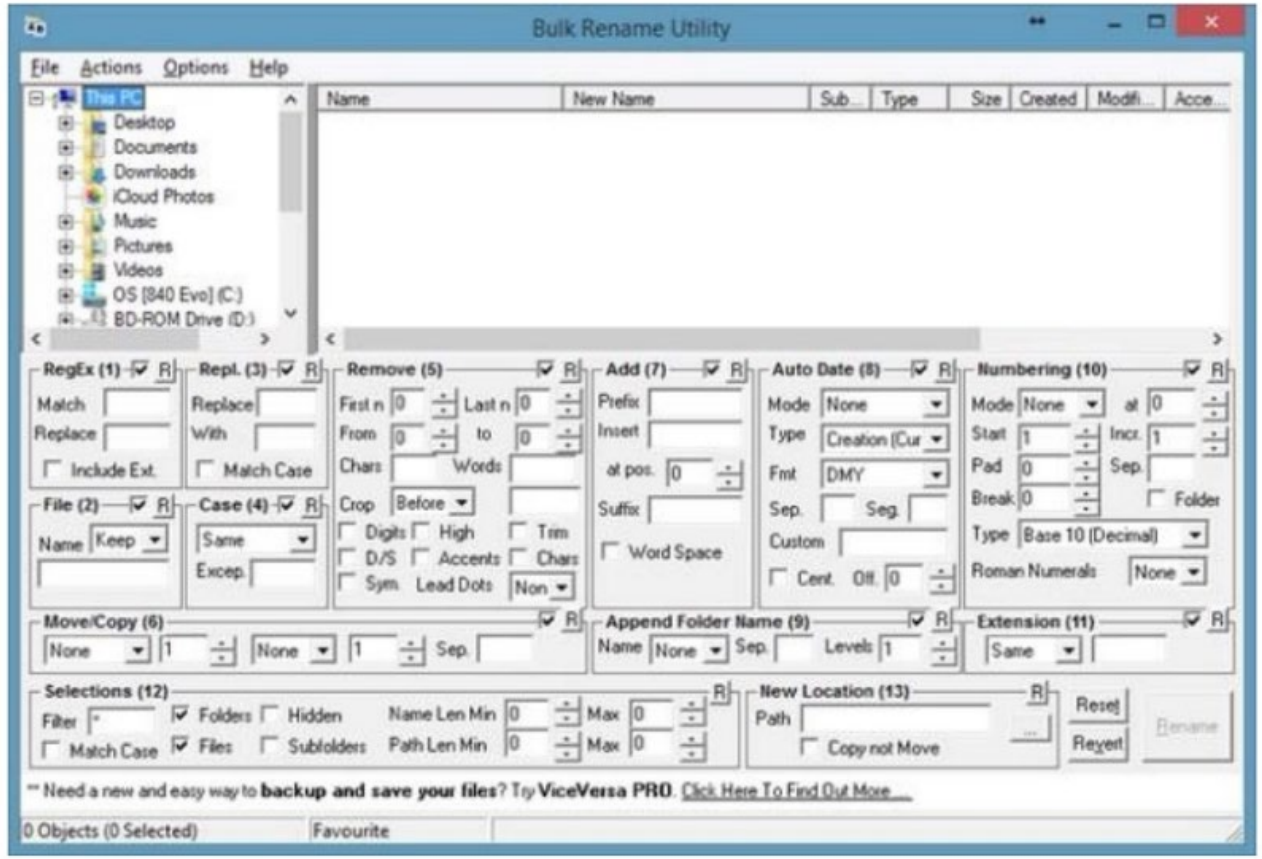

Design
Utveckling


Design
Utveckling

**Elias Betinakis**
@eliasbetinakis

Following

# why designers exist




RETWEETS **46**

FAVORITES **26**

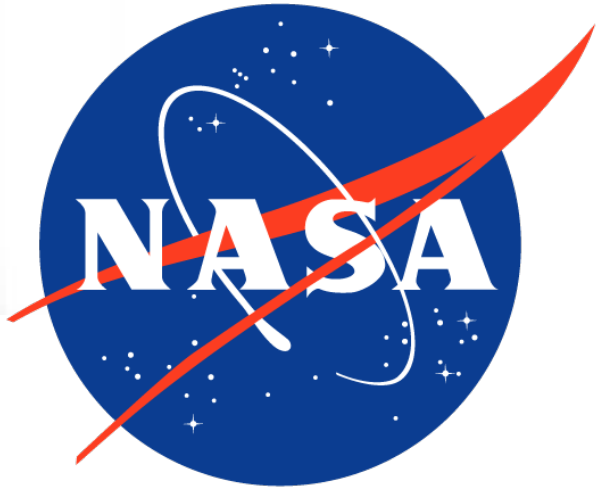
1:29 PM - 28 Oct 2014


Google


NETFLIX


Spotify®



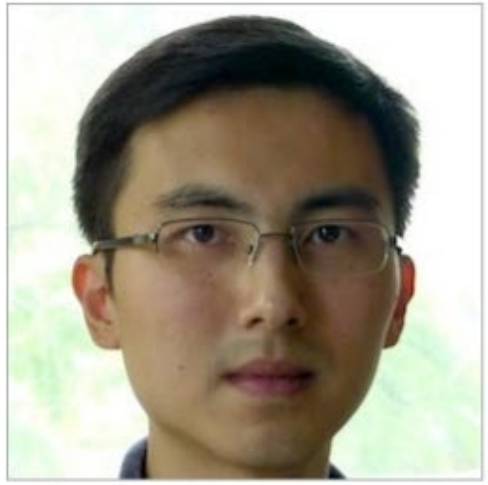

NASA

BLOG@CACM

# Python is Now the Most Popular Introductory Teaching Language at Top U.S. Universities

By Philip Guo
July 7, 2014
Comments (7)





## Summary

At the time of writing (July 2014), Python is currently the most popular language for teaching introductory computer science courses at top-ranked U.S. departments.

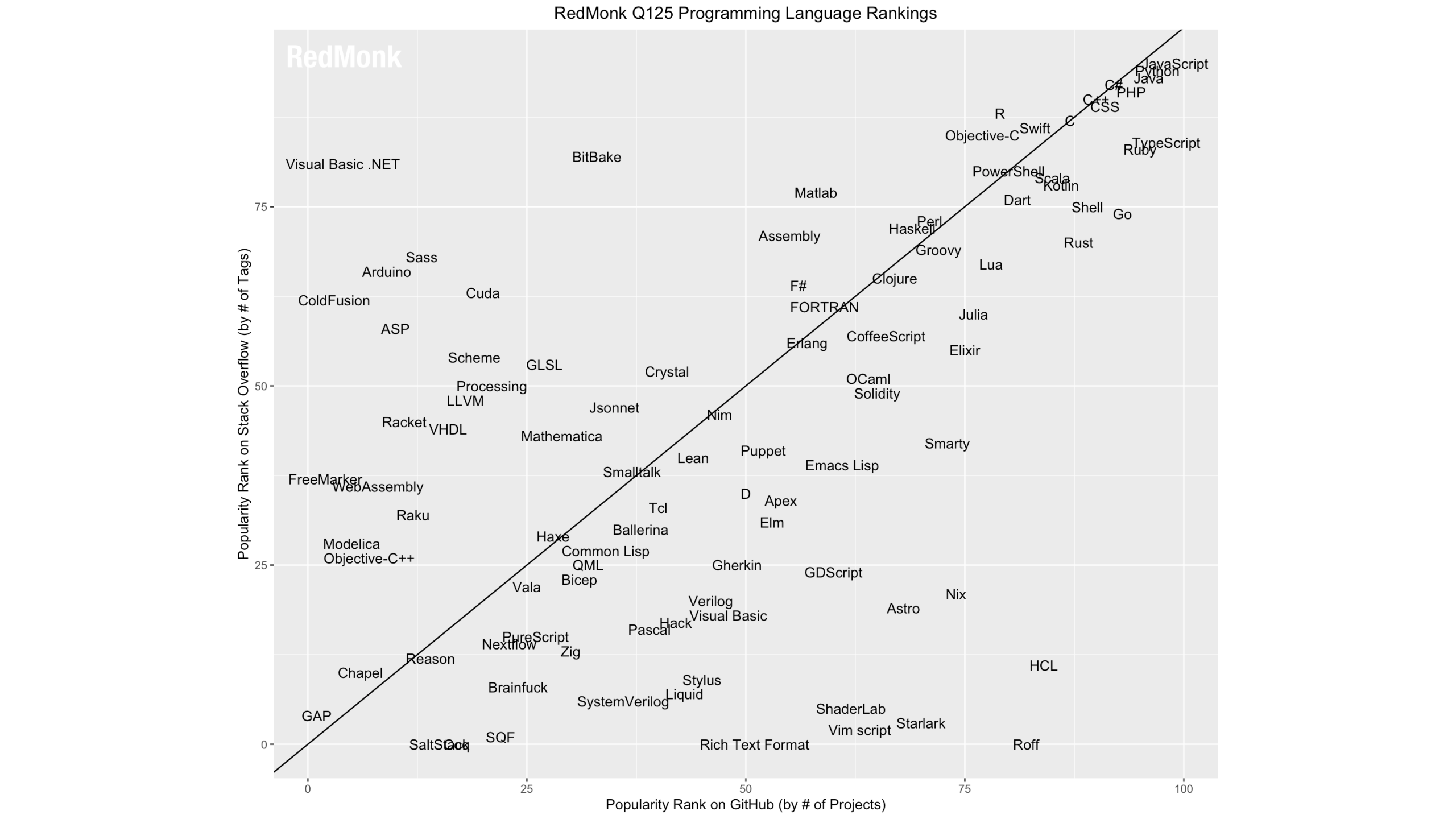
Specifically, eight of the top 10 CS departments (80%), and 27 of the top 39 (69%), teach Python in introductory CS0 or CS1 courses.

## Motivation

Python has been getting more popular as the first language to teach novices. Three years ago, Mark Guzdial blogged about the rise of Python as a teaching language and

SIGN IN for Full Access
User Name
Password
» Forgot Password?
» Create an ACM Web Account
SIGN IN

MORE NEWS & OPINIONS
Virtual Earth Plays Out Fate of Life on the Planet
New Scientist
Python is Now the Most Popular Introductory Teaching Language at Top U.S. Universities
Philip Guo


RedMonk Q125 Programming Language Rankings
RedMonk
Popularity Rank on Stack Overflow (by # of Tags)
Popularity Rank on GitHub (by # of Projects)
0
25
50
75
100
JavaScript
Python
Java
C#
PHP
C++
CSS
R
Swift
C
Objective-C
TypeScript
Ruby
PowerShell
Scala
Kotlin
Dart
Shell
Go
Perl
Haskell
Rust
Groovy
Lua
Clojure
F#
FORTRAN
Julia
CoffeeScript
Erlang
Elixir
OCaml
Solidity
Nim
Smarty
Puppet
Lean
Emacs Lisp
Smalltalk
D
Apex
Tcl
Elm
Ballerina
Haxe
Common Lisp
QML
Gherkin
GDScript
Bicep
Vala
Nix
Verilog
Visual Basic
Astro
Hack
Pascal
PureScript
Nextflow
Zig
Reason
HCL
Chapel
Stylus
Brainfuck
Liquid
SystemVerilog
ShaderLab
Starlark
Vim script
GAP
SQF
Rich Text Format
Roff
Visual Basic .NET
BitBake
Matlab
Assembly
Sass
Arduino
Cuda
ColdFusion
ASP
Scheme
GLSL
Crystal
Processing
LLVM
Jsonnet
Racket
VHDL
Mathematica
FreeMarker
WebAssembly
Raku
Modelica
Objective-C++

The TIOBE Programming Community index is an indicator of the popularity of programming languages. The index is updated once a month. The ratings are based on the number of skilled engineers world-wide, courses and third party vendors. Popular web sites Google, Amazon, Wikipedia, Bing and more than 20 others are used to calculate the ratings. It is important to note that the TIOBE index is not about the *best* programming language or the language in which *most lines of code* have been written.

The index can be used to check whether your programming skills are still up to date or to make a strategic decision about what programming language should be adopted when starting to build a new software system. The definition of the TIOBE index can be found here.

| Nov 2026 | Nov 2025 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| 1 | 1 | | Python | 20.86% | -2.51% |
| 2 | 2 | | C | 11.48% | +1.80% |
| 3 | 4 | ↑ | Java | 8.89% | +0.34% |
| 4 | 3 | ↓ | C++ | 8.35% | -0.60% |
| 5 | 5 | | C# | 3.39% | -4.26% |
| 6 | 6 | | JavaScript | 2.42% | -1.00% |
| 7 | 12 | ⇈ | R | 2.27% | +0.60% |
| 8 | 10 | ↑ | SQL | 1.91% | +0.11% |
| 9 | 7 | ↓ | Visual Basic | 1.88% | -1.43% |
| 10 | 11 | ↑ | Go | 1.69% | -0.03% |
| 11 | 14 | ↑ | Rust | 1.63% | +0.25% |
| 12 | 16 | ⇈ | PHP | 1.58% | +0.27% |
| 13 | 13 | | Fortran | 1.21% | -0.34% |
| 14 | 19 | ⇈ | Scratch | 1.15% | +0.13% |

40k Shares

# PYPL PopularitY of Programming Language

**Worldwide**, Nov 2026 :

| Rank | Change | Language | Share | 1-year trend |
|---|---|---|---|---|
| 1 | | Python | 52.04 % | +23.6 % |
| 2 | ↑ | Java | 12.56 % | +0.2 % |
| 3 | ↓ | C/C++ | 8.49 % | -6.3 % |
| 4 | | R | 4.35 % | -2.4 % |
| 5 | ↑↑↑↑↑↑↑ | Objective-C | 3.36 % | +1.5 % |
| 6 | ↓ | JavaScript | 2.87 % | -2.8 % |
| 7 | ↑↑↑ | Ada | 2.07 % | -0.8 % |
| 8 | | PHP | 1.74 % | -1.6 % |
| 9 | | Rust | 1.74 % | -1.1 % |
| 10 | ↓↓↓↓ | Swift | 1.63 % | -2.4 % |
| 11 | ↓↓↓↓ | C# | 1.54 % | -1.8 % |
| 12 | ↓ | TypeScript | 1.44 % | -0.5 % |
| 13 | | Matlab | 0.86 % | -0.8 % |
| 14 | ↑↑↑ | Ruby | 0.79 % | -0.3 % |
| 15 | ↑↑↑↑↑ | Go | 0.6 % | -0.2 % |
| 16 | ↓↓ | Powershell | 0.54 % | -0.8 % |

The PYPL PopularitY of Programming Language Index is created by analyzing how often language tutorials are searched on Google.

The more a language tutorial is searched, the more popular the language is assumed to be. It is a leading indicator. The raw data comes from Google Trends.

If you believe in collective wisdom, the PYPL Popularity of Programming Language index can help you decide which language to study, or which one to use in a new software project.



# Programmera program

# Att skapa ett program

- Uppgiftsformulering, vad är det för uppgift som ska lösas?
- Vilka steg behöver utföras för att lösa uppgiften?
- Vilka instruktioner kan användas för att utföra varje delsteg (algoritm)

# Algoritmer

- En beskrivning över hur man löser ett problem. Algoritmen består av ett antal instruktioner och beskriver i vilken ordning instruktionerna ska utföras.



PASTA CARBONARA

SKRIV UT

*Gott och klassiskt recept på pasta carbonara, men på ett lite enklare sätt.*

Tid: 25 minuter

4 portioner

INGREDIENSER:

- 300 g spaghetti
- 150 g bacon
- 1 st gul lök
- ev. 1 klyfta vitlök
- 2 st ägg
- 0.5 dl grädde
- 100 g riven parmesanost
- salt
- grovmalen svartpeppar



*Spaghetti carbonara är väldigt gott. Det här är ett enkelt (utan krångel med äggulor) och mycket gott recept på denna goda rätt.*

Dela: facebook twitter Mejla

GÖR SÅ HÄR:

Koka spaghettin enligt förpackningens anvisningar, men med 1 tsk olivolja i vattnet.

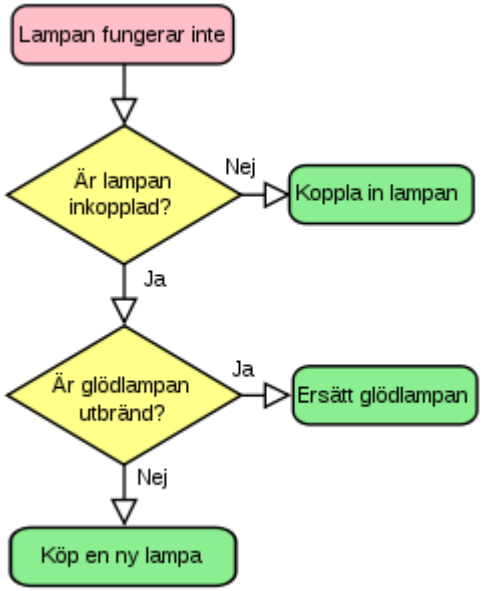
Skär bacon i små bitar. Hacka den gula löken fint. Om du vill ha vitlök, hacka vitlöksklyftan fint.

Hetta upp en stekpanna på medelhög värme och stek bacon och lök var för sig tills baconet blir knaperstekt och löken genomstekt. Sänk ev. temperaturen på plattan lite så att det inte blir bränt.

Vispa under tiden ihop ägg, grädde och riven ost. Använd en vanlig visp och vispa bara lätt upp äggen. Smaksätt med salt.

Häll av vattnet från den färdigkokta pastan och blanda med knaperstekt bacon och lök. Blanda sedan ihop med äggblandningen. Rör kraftigt med en sked. Servera på tallrik och strö över grovmalen svartpeppar och gärna lite persilja.

http://www.recepten.se/recept/pasta_carbonara.html




http://upload.wikimedia.org/wikipedia/commons/thumb/3/3c/LampFlowchart-sv.svg/250px-LampFlowchart-sv.svg.png

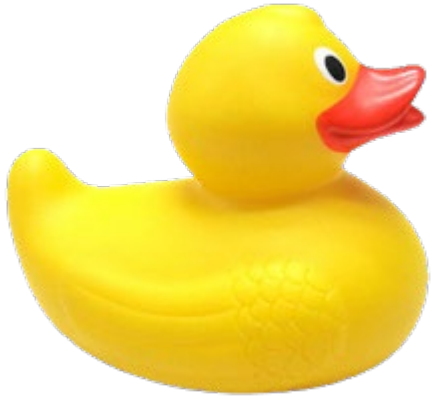
# Att tänka på (1)

- **Uppgiftsformulering**, vad är det för uppgift som ska lösas? Formulera uppgiften i termer av vad en dator kan utföra. Avgränsa problemet, vad är en del av uppgiften? Vad ingår inte?
- **Algoritmkonstruktion**, vilka algoritmer är de mest lämpliga för detta problem? Konstruera strukturen på programmet och skriv ner så kallad pseudokod. Detta är kreativ problemlösning.
- **Kodning**, översätt pseudokoden till ett programmeringsspråk t.ex. JavaScript eller Python
- **Dokumentation**, beskriva din lösning både i löpande text, med hjälp av UML och som kommentarer i programmet.

# Att tänka på (2)

- **Verifikation**, är programmet byggt på ett bra sätt så att det löser uppgiften utan att fel uppstår och det är lätt att underhålla.
- **Validering**, är användaren nöjd med hur programmet fungerar. Underhåll, åtgärda buggar, förbättra och lägg till funktionalitet.
- **Underhåll**, åtgärda buggar, förbättra och lägg till funktionalitet.

# Johans tips





# Johans tips

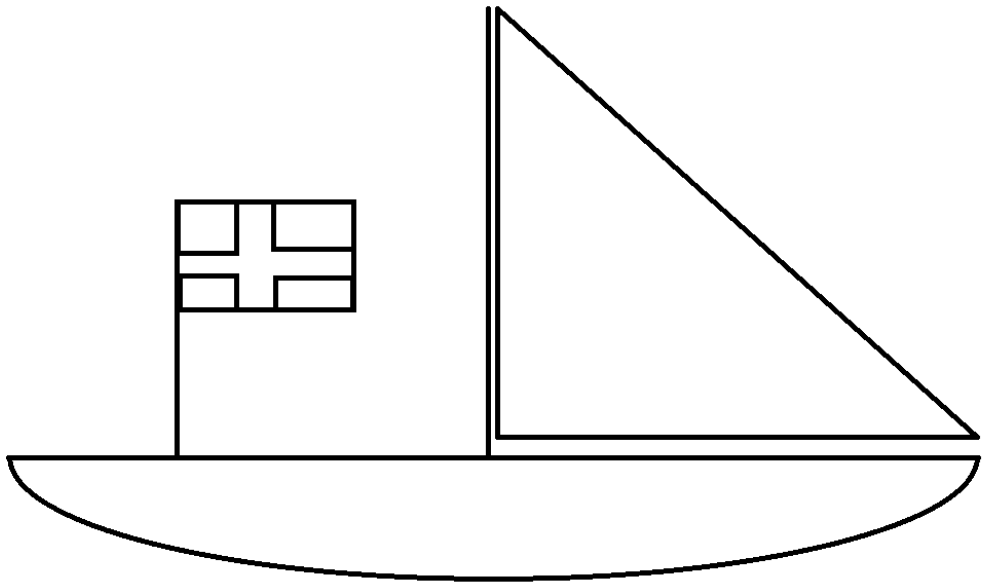


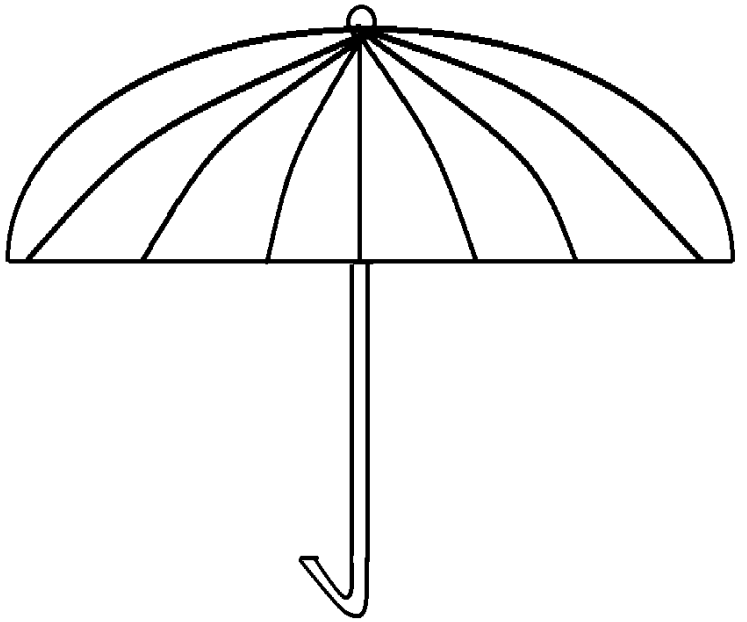




# Övning!

# Skriv instruktioner för att rita följande





OBS – Du får **INTE** använda ord som ”Segelbåt”, ”Paraply”, ”Flagga”, etc. utan enbart symboler so t.ex. ”Streck”, ”Halvcirkel”, ”Rektangel”, etc. Den som ritar **får inte prata / ställa frågor.**

# Tillbaka till Python

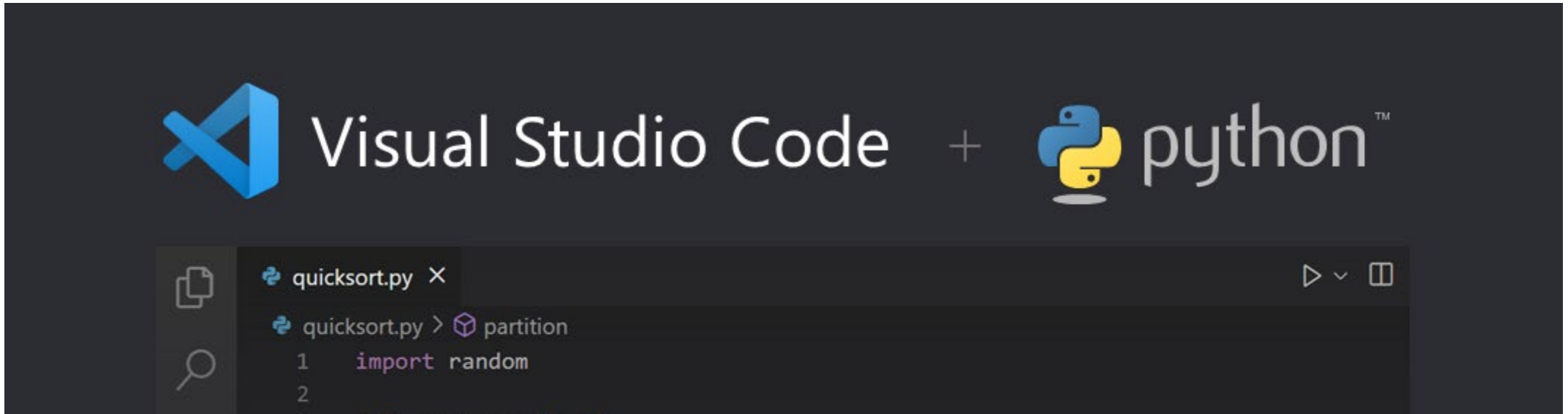


*”**Python** är ett programspråk som utformades i slutet av 1980-talet av Guido van Rossum. Python har utvecklats till ett kraftfullt och samtidigt smidigt språk med stöd för flera olika programmeringsparadigmer, bland annat objektorienterad och funktionell programmering. Till språket hör ett rikt standardbibliotek.”*

https://sv.wikipedia.org/wiki/Python_(programspråk)

# Python - utvecklingsmiljö

- Vi kommer att arbeta i **VS Code**
  - Kan laddas ner här: https://code.visualstudio.com/
  - Instruktioner för att komma igång finns på vår kurswebbplats: https://mau-webb.github.io/resurser/da354b-ht26/vs-code/
- Man kan använda valfri utvecklingsmiljö, t.ex. andra textredigerare
  - Men då får man exekvera sin kod från terminalen/konsolen istället



# Datatyper?

- För att Python ska veta vad för typ av data som hanteras behöver vi delge denna information till Python

- De olika datatyperna som vi kommer att titta på är:
  - **Number (int, float)**
  - **String**
  - Boolean
  - List
  - Tuple
  - Dictionary

# Räkna med Python

- Python är väldigt bra på att räkna, vilket vi kan göra genom vanliga matematiska uttryck.
- När vi räknar och gör matematiska operationer använder vi oss utav datatypen **number** (numerisk datatyp). Här kan vi räkna med heltal.
- Vill vi istället räkna med decimaler använder vi datatypen **float**

```
>>> 5 + 5
10
>>> 10 * 2
20
>>> 20 / 4
5
```

# Operander i Python

| Namn | Operand | Beskrivning |
|---|---|---|
| Addition | + | Beräknar summan av två tal |
| Subtraktion | - | Beräknar differensen mellan två tal |
| Multiplikation | * | Beräknar produkten av två tal |
| Division | / | Beräknar kvoten av två tal |
| Heltalsdivision | // | Beräknar heltalskvoten av två tal |
| Rest | % | Beräknar resten efter en division av två heltal |
| Exponent | ** | (x ** y) Beräknar x upphöjt till y $x^y$ |

https://sv.wikipedia.org/wiki/Python_(programspråk)

# Hantera text med Python

- Vill vi skriva ut text i Python så använder vi datatypen **string** (textsträng). Tänk på att textsträngar är just bara text – d.v.s. tecken uppradade efter varandra.
  - Vi kan därför t.ex. inte räkna med strängar
- Strängar skrivs m.h.a. ’ eller ”

```
>>> print("Hello World")
Hello World
>>> print("Tjena kexet, sitter du här och smular?")
Tjena kexet, sitter du här och smular?
```

```
>>> input("Vad heter du?")
Vad heter du?Anton
'Anton'
>>> input("Vilket är Sveriges bästa fotbollslag?")
Vilket är Sveriges bästa fotbollslag?Elfsborg
'Elfsborg'
```

# Lägga ihop strängar med varandra

- Detta vill man ofta göra för att bygga upp meddelande till användaren av ens program. T.ex.

```
name = input("Vad heter du? ")
print("Välkommen " + name)
```

```
Vad heter du? Anton
Välkommen Anton
```

- Vi tar alltså de strängarna som står på varje sida av ”+” och sammanfogar dessa.
- Skulle vi försöka med räkna mer strängar får vi alltså inte önskat resultat. T.ex.

```
>>> "5" + "7"
'57'
```

```
>>> "5" * 4
'5555'
```

# Variabler

Spara information

# Variabler

- När man bygger ett program vill man gärna spara undan värde, så att vi kan använda dem vid ett senare tillfälle.
  - Hur kul vore det om man glömde allt – hela tiden?
- Detta gör man genom att använda **variabler**
  - Variabler är ett namn som refererar till ett värde (kan vara av vilken datatyp som helst)
- Man skapar en variabel genom att
  1. Välja ett lämpligt namn
  2. Tilldela ett lämpligt värde genom ”=”-tecknet

# Ange datatyper?

Python listar ut detta beroende värde

# Kommentarer

- Det är en god idé att dokumentera sin kod
  - Man glömmer bort vad koden gör
  - Man glömmer bort hur koden gör det
  - Man glömmer bort varför koden gör det
- Man kommenterar sin kod genom #

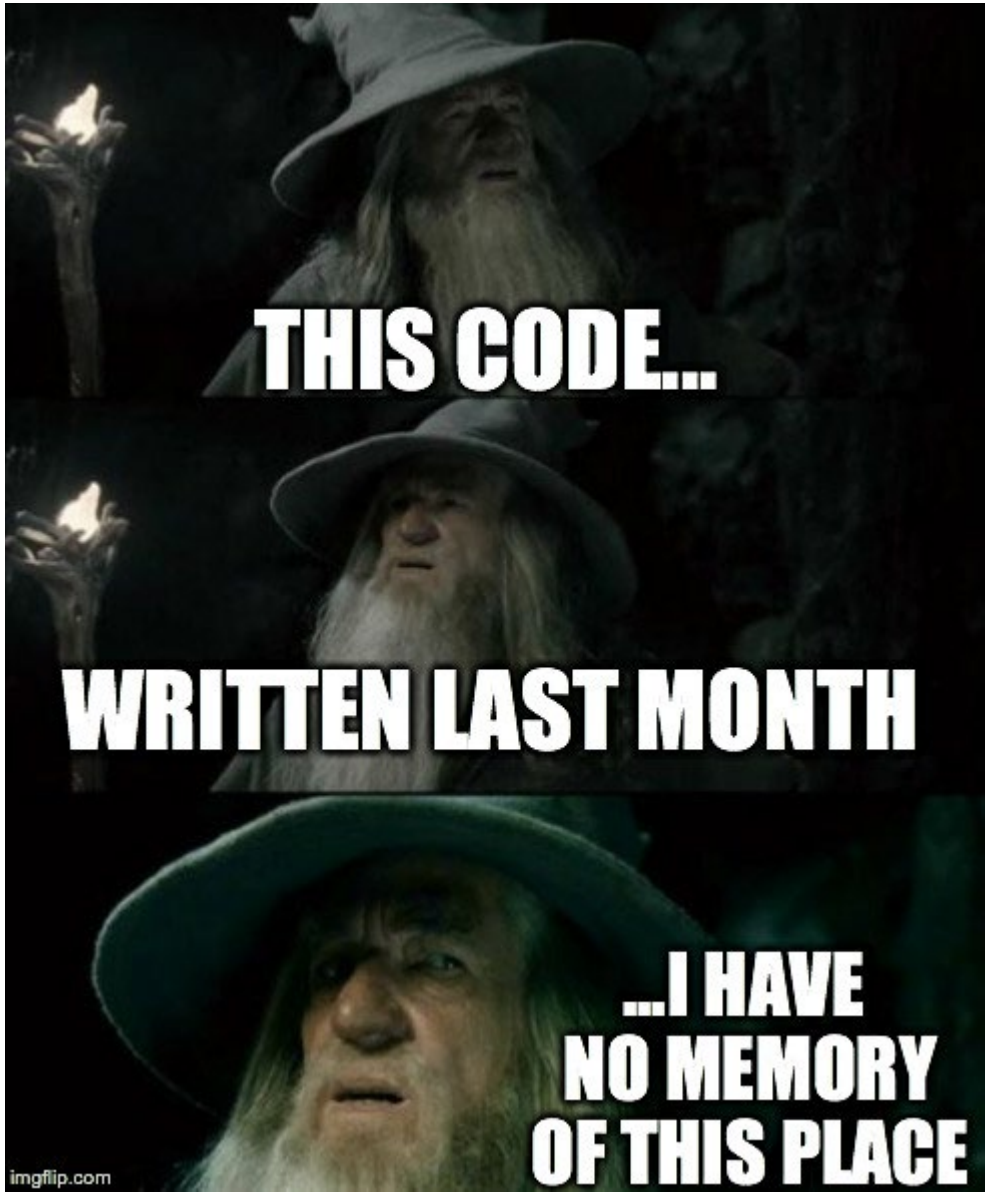
```
# Denna rad kommer inte visas, utan finns bara till för oss

# 1) Frågar efter användarens namn, och sparar det i variabeln "name"
name = input("Vad heter du? ")

# 2) Frågar efter användarens ålder, och sparar det i variabeln "age"
age = input("Hur gammal är du? ")

# 3) Skriver ut ett välkomnstmeddelande
print("Hej " + name + " (" + age + "år)!")
```

```
Vad heter du? Anton
Hur gammal är du? 26
Hej Anton (26år)!
```




IS IT MAGIC?
DEMO TIME
memegenerator.net